

The Xenomai Project, a Linux Based RTOS

Real-time operating systems (RTOS) are designed for real-time applications such as embedded systems, industrial robots, scientific research equipment and others. Read on to discover more about the Xenomai project, which emulates a traditional RTOS, but is ported to GNU/Linux.

Today, Linux is used a lot in embedded products and industrial applications. But sometimes, even Linux is not competent enough for embedded systems as they need some specific implementation practices. So this need has given birth to the real-time operating system (RTOS). Mostly, RTOS applications are embedded so they are used in embedded products/systems. They typically have a small footprint.

Xenomai history

The Xenomai project was started in 2001, with the aim of emulating a traditional RTOS and facilitating its porting to GNU/Linux. Xenomai was initially related to RTAI (Real-Time Application Interface) but now it is independent.

The Xenomai core running in the kernel space is offered under GPL 2. The user space interface libraries are released under the LGPL 2.1. Xenomai supports many architectures like PowerPC, Blackfin, ARM, x86, x86_64, etc.

Xenomai architecture

So to give a real-time guarantee to the applications, Xenomai real-time sub-system is patched with the Linux kernel. It's basically a dual kernel approach in which both kernels work side by side without one kernel being aware that the other kernel is working in parallel see Figure 1. Both can communicate with each other via a virtual interrupt controller or I-pipe.

So there are two kernels:

- 1) A Linux kernel: For non-real-time tasks, and
- 2) The Xenomai kernel: For real-time tasks.

The Xenomai kernel gives predicted results hence it is also known as the primary mode while the Linux kernel does not give predicted results and hence is known as the secondary mode. Real-time applications are expected to work in primary mode while other applications are expected to work in secondary mode. Switching between modes takes place automatically.

This article talks about a real-time implementation of Linux known as the Xenomai project.

The main goal of the Xenomai project is to facilitate the migration of industrial applications from the proprietary world to a GNU/Linux based environment, while keeping real-time guarantees.

The interrupt pipeline

This is software between the hardware, Linux and Xenomai that acts as a virtual programmable interrupt controller

Interrupt pipeline(I-pipe) is a software layer which maintains the policies for handling interrupts and masking them. We can say that both the Linux and Xenomai domains

are connected to each other. Domains are usually kernel modules that call some I-pipe service.

Hardware abstraction layer (HAL)

Xenomai has its own HAL which is needed to port Xenomai to a particular processor architecture. All skins and core services are on top of the HAL.

The Xenomai core and nucleus

The Xenomai core is responsible for supplying operating system service to skins which mimic traditional RTOS APIs. It is because of the Xenomai core that developers get predictable latencies. You can imagine the Xenomai core as an abstract RTOS. These building blocks are gathered into a single, loadable module called the Xenomai nucleus.

Xenomai skins

The various types of skins supported by Xenomai are listed below:

- POSIX
- VxWorks
- pSOS+
- VRTX
- uITRON
- RTAI 3.x
- Native API
- RTDM (Real-Time Driver Model)

The real-time driver model

Developing a real-time driver for a dual kernel architecture would be a difficult task if there is no common framework available for it. Hence the RTDM (real-time driver model) skin gives that common framework to easily port the real-time driver see Figure 2. There are two classes of RTDM:

- 1) The protocol driver, which connects to a socket interface and hence is well suited to managing message-oriented communications with a real-time device, and...
- 2) A named device, which is like a character driver in the Linux device driver.

Implementation

For the purposes of implementation, I am using a I2C driver developed by me for Beagle-Bone Black with Linux-3.8.13 and Xenomai 2.6.3. The full I2C driver code is available at https://github.com/JayKothari/i2c_rtdm. The driver was tested with an application made for a I2C sensor (BMP085 pressure sensor).